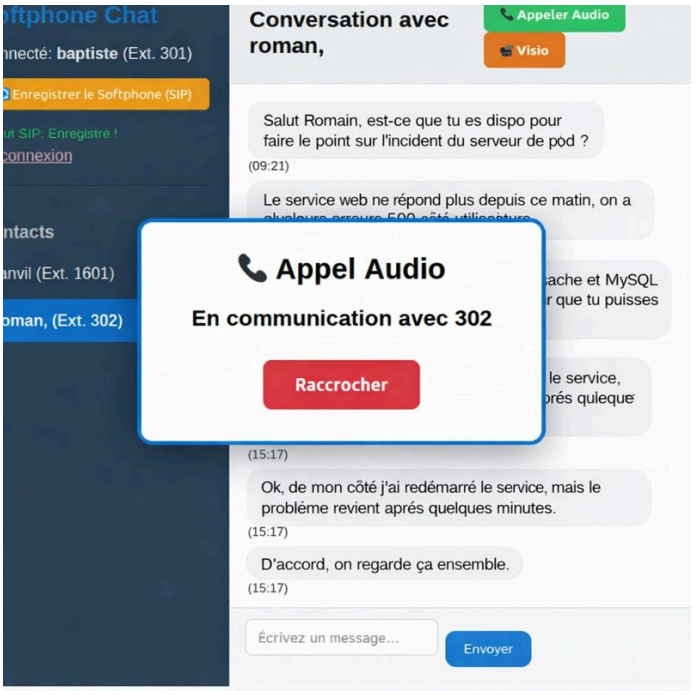
SAE 503 :
Déployer et gérer les services ROM

Groupe :
Baptiste Duval, Michel Bauchart, Roman Negrila



I. INTRODUCTION

Le projet s'inscrit dans le cadre de l'unité d'enseignement R5 ROM.09 et de la Saé 5 ROM.03 (Déployer et gérer les services ROM) au sein du département R&T de l'IUT de Béthune.

L'objectif principal est de mettre en pratique les compétences d'un professionnel spécialisé en Réseaux Opérateurs et Multimédia. Ce dernier doit être capable de déployer et d'administrer des services variés tels que la téléphonie IP et les services vidéo pour le compte d'entreprises ou d'opérateurs. Ce projet utilise des techniques de virtualisation (via la distribution Alpine Linux) pour optimiser le déploiement de ressources multimédias dans un environnement réseau.

Le déroulement du projet est structuré en neuf étapes clés, allant du déploiement initial d'un serveur de communication à la réalisation d'un audit de sécurité complet des échanges.

II. OBJECTIF DU PROJET

La thématique centrale du projet est l'intégration des services de communication d'un **autocommutateur Asterisk** au sein d'une **interface web personnalisée**. Les objectifs se divisent en plusieurs axes de développement :

- **Intégration WebRTC** : Développer une application web permettant de réaliser des échanges voix et vidéo entre deux terminaux à l'aide de la librairie **sip.js**.

- **Administration et Services** : Mettre en place un serveur Asterisk sur Alpine, configurer des comptes pour terminaux IP, et gérer l'administration du serveur via une interface HTTP simple.

- **Communications Unifiées** : Compléter l'interface pour inclure des fonctionnalités modernes telles qu'une **messagerie instantanée** et un portail dédié aux **visioconférences**.

- **Analyse et Sécurité** : Valider techniquement les échanges à l'aide d'outils d'analyse réseau comme Wireshark et réaliser un **audit de sécurité** pour prévenir les risques d'interception (type Man-In-The-Middle) entre le portail web et le serveur Asterisk.

III. ARCHITECTURE DU PROJET

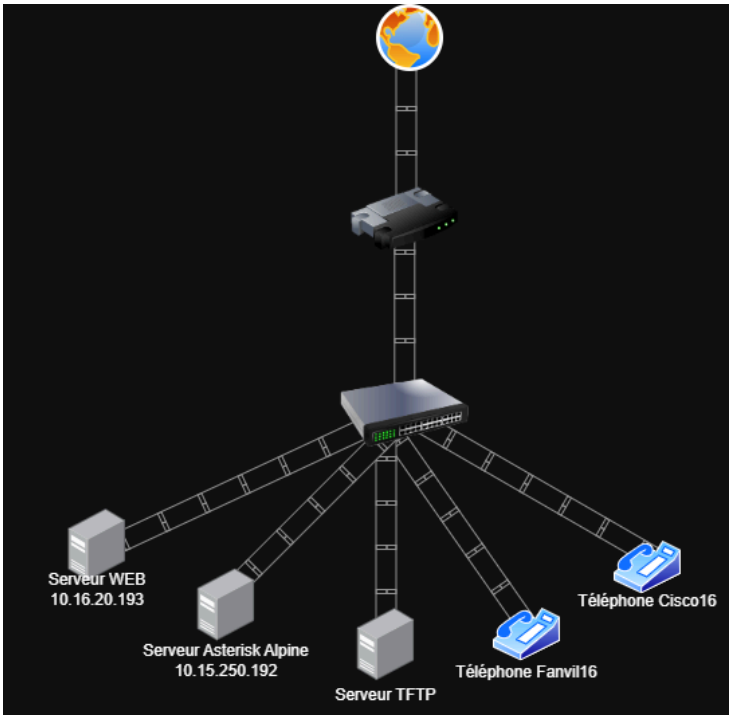
Dans ce projet nous avons donc 3 machines virtuelles :

- **Serveur WEB,** création d'une interface graphique style Teams, sur laquelle nous pourrons envoyer des messages en instantané, effectuer des visios conférences et pouvoir appeler d'autres personnes.

- **Serveur TFTP,** ce serveur nous sert uniquement à renseigner et pouvoir utiliser notre téléphone Cisco.

- **Serveur ASTERISK,** Ce serveur va nous servir à configurer les téléphones (extensions, messagerie vocale), il va nous permettre aussi de configurer les utilisateurs du Softphone (enregistrement SIP, appel, messagerie).

Nous avons aussi deux téléphones, un téléphone Fanvil et un téléphone Cisco avec lequel nous pourrons nous appeler.



Architecture du projet

IV. Étape 1

Déployer un serveur Asterisk sur une machine virtuelle avec la distribution Alpine

Pour configurer le téléphone Fanvil il faut tout d'abord créer une machine Asterisk, il faut ensuite se connecter à la machine et démarrer le service asterisk :

`service asterisk start`	`Pour démarrer le service`

`service asterisk status`	`rt-mu:~# service asterisk status`
	`* status: started`
	`rt-mu:~# _`

V. Étape 2

`Téléphone Fanvil :`

Ensuite il faut installer les paquets suivant :
`apk add nano`
`apk add openssh`

Maintenant il faut créer le compte du téléphone dans le serveur :

`cd /etc/asterisk/`	`on se place dans le dossier`
`cp pjsip.conf pjsip.conf.src`	`Sauvegarde du fichier source`
`nano pjsip.conf`	`Création de notre fichier vierge`

Tout d'abord dans ce fichier nous devons configurer le transport des données, ici en udp.

```
[transport-udp]
type=transport
protocol=udp
bind=0.0.0.0:5060
```

Quand nous avons fait cela nous pouvons configurer le compte du téléphone, ici Fanvil16 car nous occupons le poste 16.

```
[Fanvil16]
type=endpoint
context=internal
auth=auth-Fanvil16
aors=Fanvil16
transport=transport-udp
disallow=all
allow=ulaw
mailboxes=1601@default

[auth-Fanvil16]
type=auth
auth_type=userpass
password=12345
username=Fanvil16

[Fanvil16]
type=aor
max_contacts=1
```

Puis quand nous avons fait cela il nous faut configurer les extensions.

`cd /etc/asterisk/`	`on se place dans le dossier`
`cp extensions.conf extensions.conf.src`	`Sauvegarde du fichier source`
`nano extensions.conf`	`“Création de notre fichier vierge”`

Maintenant nous pouvons configurer son extension.

```
[general]
static=yes
writeprotect=no

[internal]

; Extension pour le téléphone Fanvil (1601)
exten => 1601,1,NoOp(Appel vers le poste Fanvil)
exten => 1601,n,Dial(PJSIP/Fanvil16,12)
exten => 1601,n,VoiceMail(1601@default,b)
```

```
exten => 1601,n,Hangup()
```

Maintenant que la configuration des fichiers est terminée, nous pouvons redémarrer le serveur asterisk pour que la nouvelle configuration soit prise en compte.

`service asterisk restart`	`Pour redémarrer le service`

Pour faire cela, nous devons d'abord réinitialiser le téléphone.

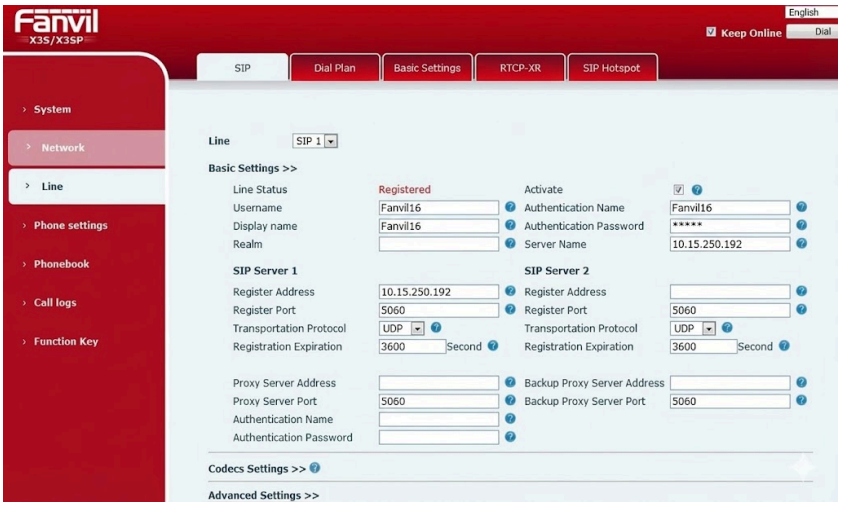
- Déconnecter la fiche RJ45 votre téléphone
- Maintenir la touche # et connecter la fiche RJ45 sur le port du téléphone.
- Attendre l'apparition du message *Post Mode*
- Appuyer successivement sur les touches *#168
- Attendre l'apparition du message *Phone reset*
- Déconnecter et reconnecter la fiche RJ45 sur le port du téléphone.

Quand ça sera fait le téléphone récupère une adresse IP automatiquement, il suffit de récupérer l'adresse IP et la mettre sur le navigateur.

Cela nous demandera des identifiant il nous suffit mettre :

Username	***admin***
password	***admin***

Puis aller dans la page Line et remplir comme ici :



Quand c'est écrit Registered c'est que la configuration est bien correcte et que le téléphone est enregistrer et configurer.

TÉLÉPHONE CISCO :

Pour configurer le téléphone Cisco le principe est le même au niveau du serveur asterisk.

`cd /etc/asterisk/`	`on se place dans le dossier`
`nano pjsip.conf`	`Lecture du fichier pour le modifier`

Puis nous ajoutons la configuration du Cisco16 :

```
[Cisco16]
type=endpoint
context=internal
auth=auth-Cisco16
aors=Cisco16
transport=transport-udp
disallow=all
allow=ulaw
mailboxes=1602@default

[auth-Cisco16]
type=auth
auth_type=userpass
password=12345
username=Cisco16

[Cisco16]
type=aor
max_contacts=1
```

Maintenant que nous avons configuré son compte nous pouvons modifier son extension pour finir sa configuration dans le serveur Asterisk.

`cd /etc/asterisk/`	`on se place dans le dossier`
`nano extensions.conf`	`Lecture du fichier pour le modifier`

Et nous y ajoutons en dessous de la configuration du Fanvil16 :

```
; Extension pour le téléphone Cisco (1602)
exten => 1602,1,NoOp(Appel vers le poste Cisco)
exten => 1602,n,Dial(PJSIP/Cisco16,12)
exten => 1602,n,VoiceMail(1602@default,u)
exten => 1602,n,Hangup()
```

Puis maintenant nous pouvons redémarrer le serveur Asterisk pour prendre en compte les modifications.

`service asterisk restart`	`Pour redémarrer le service`

Quand nous avons fait cela, nous pouvons créer le serveur TFTP, quand nous serons connectés à la machine nous avons plusieurs pré-requis à faire.

Installer et démarrer les services utiles :

`apk add tftp-hpa`	`Installation du service tftp`
`service in.tftpd start`	`Démarrage du service tftp`
`apk add openssh`	`Installation du service openssh`
`service sshd start`	`Démarrage du service openssh`
`apk add nano`	`Installation de nano pour éditer les fichiers`

Quand nous avons fait cela il nous faut noter l'adresse IP du serveur "XXX.XXX.XX.XX", puis se placer dans le répertoire /var/tftpboot/ donc `cd /var/tftpboot/` et faire cette commande wget http://iut-rt.univ-artois.fr/tph/Archive.zip et décompresser le dossier, ce qui nous donnera deux fichiers qui vont être utile à la configuration.

Nous allons commencer par modifier le fichier `SEP_MAC-CISCO_.cnf.xml` en le renommant et en remplaçant `_MAC-CISCO_` par l'adresse MAC du téléphone.

`mv SEP_MAC-CISCO_.cnf.xml SEPXXXXXXXX.cnf.xml`	`Renomme le fichier`
`nano SEPXXXXXXXX.cnf.xml`	`Modification du fichier`

Dans ce fichier nous allons modifier les lignes suivantes.

Ancienne configuration :

```
<processNodeName>10.16.19.249</processNodeName>

<phoneLabel>Cisco01</phoneLabel>
 <featureLabel>Cisco01</featureLabel>
 <authName>Cisco01</authName>
 <contact>Cisco01</contact>
```

Nouvelle configuration :

```
<processNodeName>10.15.250.191</processNodeName>
<phoneLabel>Cisco16</phoneLabel>
 <featureLabel>Cisco16</featureLabel>
 <authName>Cisco16</authName>
 <contact>Cisco16</contact>
```

Puis relancer le serveur en faisant :

```
service in.tftpd stop
service in.tftpd start
```

Il nous reste plus qu'à aller sur le téléphone dans les **Settings > Admin Settings > Network Setup > IPv4 Setup** puis nous cherchons **Alternate TFTP** que nous allons mettre en **YES**, puis chercher **TFTP Server 1** et y mettre l'IP du serveur TFTP. Il suffit d'appliquer les changements et le téléphone sera paramétré.

A partir de ce moment nous allons pouvoir appeler le téléphone Fanvil depuis le téléphone Cisco et inversement.

https://drive.google.com/file/d/1b5jJr3H5x2karbUq60i0BJJoSmtbQw87/view?usp=sharing

Lien de la vidéo pour l'appel entre les deux téléphones.

Nous avons pris les numéros 1601 et 1602 car nous sommes au poste 16.

Mise en place de la boîte vocale :

Modification du fichier extensions.conf pour mettre en place l'extension de la boîte vocale.

```
exten=>500,1,VoiceMailMain()
same=>n,Hangup()
```

Donc ici quand depuis 1602 j'appelle 1601 et que ce téléphone ne répond pas nous serons redirigé vers la messagerie vocale, nous allons enregistrer le message et depuis le téléphone 1601 écouter le message.

https://drive.google.com/file/d/1-FgEOD2Gmtaa0Hs79SybjBuZ2dCscV6m/view?usp=sharing

Echange de voix entre les deux téléphones :

Quand nous nous appelons nous pouvons voir des trames UDP et TCP reçues.

2261	36.877383032	10.16.20.193	10.15.250.192	UDP	224	39517 → 13356 Len=182
2262	36.891709743	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182
2263	36.904626952	10.16.20.193	10.15.250.192	UDP	224	39517 → 13356 Len=182
2264	36.904680116	10.16.20.193	10.15.250.192	UDP	224	39517 → 13356 Len=182
2265	36.912053947	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182
2266	36.932169991	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182
2267	36.936560617	10.16.20.193	10.15.250.192	UDP	224	39517 → 13356 Len=182
2268	36.952055140	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182
2269	36.963339167	10.16.20.193	10.15.250.192	UDP	224	39517 → 13356 Len=182
2270	36.972196285	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182
2271	36.989935078	10.16.20.193	10.15.250.192	UDP	224	39517 → 13356 Len=182
2272	36.991937900	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182
2273	37.012171171	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182
2274	37.015279263	10.16.20.193	10.15.250.192	UDP	224	39517 → 13356 Len=182
2275	37.032295108	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182
2276	37.043578979	10.16.20.193	10.15.250.192	UDP	224	39517 → 13356 Len=182
2277	37.049012364	10.16.20.193	10.15.250.192	UDP	224	39517 → 13356 Len=182
2278	37.052137812	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182
2279	37.055824088	10.15.250.192	10.16.20.193	UDP	156	13356 → 39517 Len=114
2280	37.056441926	10.15.250.192	10.16.20.193	TCP	786	5038 → 41178 [PSH, ACK] Seq=14067 Ack=1 Win=510 Len=720 TSval=2716788155 TSecr=…
2281	37.056487158	10.16.20.193	10.15.250.192	TCP	66	41178 → 5038 [ACK] Seq=1 Ack=14787 Win=501 Len=0 TSval=1061942704 TSecr=2716788…
2282	37.056442072	10.15.250.192	10.16.20.193	TCP	786	5038 → 41164 [PSH, ACK] Seq=14067 Ack=1 Win=510 Len=720 TSval=2716788155 TSecr=…
2283	37.056546557	10.16.20.193	10.15.250.192	TCP	66	41164 → 5038 [ACK] Seq=1 Ack=14787 Win=501 Len=0 TSval=1061942704 TSecr=2716788…
2284	37.056702558	10.15.250.192	10.16.20.193	TCP	785	5038 → 41178 [PSH, ACK] Seq=14787 Ack=1 Win=510 Len=719 TSval=2716788155 TSecr=…
2285	37.056710093	10.16.20.193	10.15.250.192	TCP	66	41178 → 5038 [ACK] Seq=1 Ack=15506 Win=501 Len=0 TSval=1061942704 TSecr=2716788…
2286	37.056702643	10.15.250.192	10.16.20.193	TCP	785	5038 → 41164 [PSH, ACK] Seq=14787 Ack=1 Win=510 Len=719 TSval=2716788155 TSecr=…
2287	37.056740945	10.16.20.193	10.15.250.192	TCP	66	41164 → 5038 [ACK] Seq=1 Ack=15506 Win=501 Len=0 TSval=1061942704 TSecr=2716788…
2288	37.072193713	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182

VI. Étape 3 : Serveur WEB

Nous avons commencé par créer une machine ubuntu-Alpine, sur laquelle nous allons nous connecter en ssh depuis une machine ubuntu Desktop :

```
sudo apt update
sudo apt install asterisk -y
```

`sudo apt update`	`Mise à jours`
`sudo apt install asterisk -y`	`Installation du paquet Asterisk`
`sudo systemctl start asterisk`	`Démarrage du service Asterisk`

Ensuite il faut installer notre serveur web donc Nginx, nous allons installer Flask et MariaDB:

`sudo apt update`	`Mise à jours`
`sudo apt install nginx -y`	`Installation du paquet nginx`
`sudo systemctl start nginx`	`Démarrage du service nginx`

`sudo apt install mariadb-server -y`	`Installation du paquet Mariadb`
`sudo systemctl start mariadb`	`Démarrage du service Mariadb`

`sudo apt install python3-pip python3-venv -y`	`Installation de python3`
`pip3 install flask`	`Installation du paquet flask`

Explication des codes :

app.py :

Connection à la base de données pour la gestion des utilisateurs et des messages.

```
app.config['SQLALCHEMY_DATABASE_URI'] =
'mysql+pymysql://chat_user:progtr00@localhost/softphone_chat'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

Configuration AMi pour se connecter au serveur Asterisk.

```
AMI_HOST = '10.15.250.192'
AMI_PORT = 5038
AMI_USER = 'webapp'
AMI_SECRET = '1234'
AMI_CONTEXT = 'internal'
```

Récupération des informations importantes dans la base de données comme les utilisateurs avec leurs mots de passe et leurs extensions associés. Nous récupérons aussi les messages envoyés avec la personne qui les a envoyés et toutes les informations importantes. Et une fonction où nous récupérons les conversations.

```
class Utilisateur(db.Model):
    __tablename__ = 'utilisateurs'
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(50), unique=True, nullable=False)
    password_hash = db.Column(db.String(256), nullable=False)
    extension = db.Column(db.Integer, unique=True) # Ajouté pour l'extension

class Message(db.Model):
    __tablename__ = 'messages'
    id = db.Column(db.Integer, primary_key=True)
    sender_id = db.Column(db.Integer, db.ForeignKey('utilisateurs.id'), nullable=False)
    recipient_id = db.Column(db.Integer, db.ForeignKey('utilisateurs.id'), nullable=False)
    content = db.Column(db.Text, nullable=False)
    timestamp = db.Column(db.DateTime, default=datetime.utcnow)

    sender = db.relationship('Utilisateur', foreign_keys=[sender_id],
backref='messages_sent')
    recipient = db.relationship('Utilisateur', foreign_keys=[recipient_id],
backref='messages_received')


# Fonction utilitaire pour récupérer les messages d'une conversation
def get_conversation(user_id1, user_id2):
    return Message.query.filter(
        or_(
            (Message.sender_id == user_id1) & (Message.recipient_id == user_id2),
            (Message.sender_id == user_id2) & (Message.recipient_id == user_id1)
        )
    ).order_by(Message.timestamp.asc()).all()
```

Partie AMI :

Cette partie du code constitue le pont technologique entre la téléphonie IP (Asterisk) et l'interface Web (Flask). Son rôle est d'informer instantanément l'utilisateur d'un événement

réseau.

1. L'Écouteur Réseau (Thread AMI)

Le code utilise le threading pour créer une oreille attentive qui tourne en tâche de fond sans bloquer l'application. Ce thread maintient une connexion TCP permanente avec l'AMI (Asterisk Manager Interface). Une boucle de reconconnexion automatique garantit que le lien est rétabli en cas de coupure réseau ou de redémarrage du serveur Asterisk.

2. Traduction et Filtrage des Événements

Les données brutes reçues d'Asterisk sont du texte non structuré.

- Parsing : La fonction `parse_ami_event` transforme ce flux en dictionnaires Python exploitables (clés/valeurs).
- Analyse : La fonction `handle_ami_event` surveille spécifiquement l'événement "Dial". C'est le signal envoyé par le serveur dès qu'une extension commence à en sonner une autre.

3. Notification Push (Socket.IO)

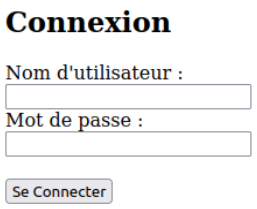
Dès qu'un appel est détecté, le serveur ne peut pas attendre que le navigateur rafraîchisse la page. On utilise alors Socket.IO pour "pousser" l'information vers l'utilisateur concerné.

- Isolation par Rooms : L'alerte est envoyée uniquement dans la "room" (chambre virtuelle) correspondant à l'extension du destinataire.
- Résultat : Le navigateur reçoit l'ordre d'afficher la fenêtre d'appel entrant en quelques millisecondes, permettant une interactivité identique à un téléphone physique.

```
--- AMI Listener Thread démarré. Tentative de connexion... ---
AMI Listener: Login réussi. Écoute des événements...
```

Partie Route :

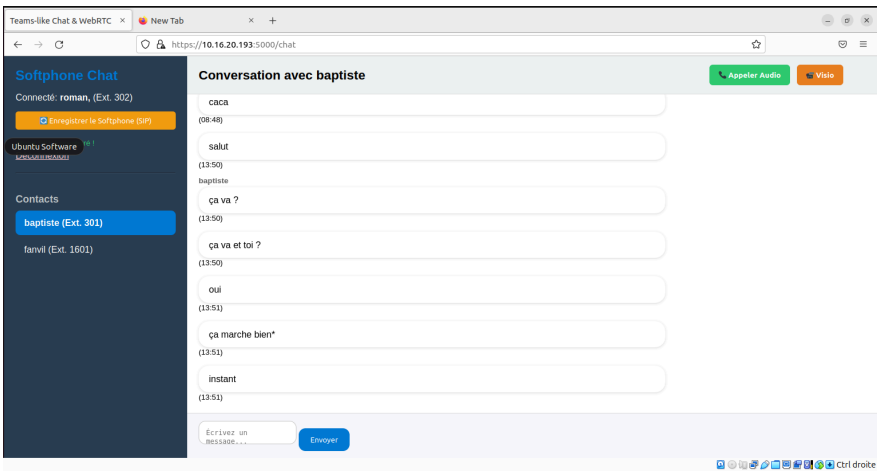
Ensuite nous avons les routes flask pour naviguer entre les pages html. Quand nous voulons nous connecter sur le site, nous devons absolument nous connecter, nous récupérons donc les informations récupérées dans la base de données. Si les identifiants sont corrects nous serons redirigé vers la page /chat.



Page de login

Quand nous sommes connectés nous arrivons sur la page /chat, où nous pouvons envoyer

des messages en instantané, on peut aussi appeler le téléphone physique.



Page d'acceuil

Nous avons donc un bouton pour pouvoir enregistrer l'extension de notre Softphone grâce à une fonction SIPRegistrer qui enverra une requête au serveur asterisk, si le téléphone s'enregistre nous pourrions appeler le téléphone physique. Les appels entre les deux comptes Softphone ne fonctionnent pas car la version du Asterisk ne le permet pas et ne fonctionne pas. Nous pouvons uniquement appeler le téléphone physique depuis le Softphone.

Configuration des extensions du SoftPhone :

Dans ce fichier nous allons créer la liste des téléphones autorisés à se connecter au serveur Asterisk.

Fichier /etc/asterisk/pjsip.conf :

```
[301]
type=endpoint
context=internal
disallow=all
allow=ulaw,alaw
aors=301
auth=301-auth
webrtc=yes
transport=transport-ws
media_encryption=dtls

[301-auth]
type=auth
auth_type=userpass
username=301
password=progtr00
realm=asterisk

[301]
type=aor
max_contacts=1
remove_existing=yes
```

```
[302]
type=endpoint
context=internal
disallow=all
allow=ulaw,alaw
aors=302
auth=302-auth
webrtc=yes
transport=transport-ws
media_encryption=dtls

[302-auth]
type=auth
auth_type=userpass
username=302
password=progtr00
realm=asterisk

[302]
type=aor
max_contacts=1
remove_existing=yes
```

Dans le fichier extensions.conf nous allons créer les numéros des utilisateurs du Softphone pour qu'ils puissent appeler les téléphones physique.

```
exten => 301,1,Dial(PJSIP/301,20)
exten => 302,1,Dial(PJSIP/302,20)

exten => _3XX,1,Dial(PJSIP/${EXTEN},20)
same => n,Answer()
same => n,Hangup()

exten => _XXXX,1,Dial(PJSIP/${EXTEN})
```

Partie AMI :

La configuration de AMI sert à ce qu'une application externe puisse piloter le serveur de téléphonie donc ici c'est pour que les appels se lancent, pour que les téléphones soient enregistrés et toutes les fonctions que nous allons utiliser.

Ici webapp sera le nom d'utilisateur et 1234 le mot de passe pour se connecter.

Configuration AMI du serveur téléphonie :

```
[webapp]
secret=1234
permit=10.16.20.193/255.255.255.255
read=system,call,log,verbose,command,agent,user,config,dtmf,reporting,cdr,diaplan,originate
write=system,call,log,verbose,command,agent,user,config,dtmf,reporting,cdr,diaplan,originate
```

Voici ce qu'il se passe quand nous enregistrons le Softphone, nous voyons dans le asterisk qu'il est bien enregistrer et que nous pouvons l'utiliser :

```
Endpoint:  301                                          Not in use    0 of inf
    InAuth:  301-auth/301
       Aor:  301                                        1
     Contact:  301/sip:ln5u6023@10.31.25.96:37618;transpo c33c4f0b8c NonQual         nan
 Transport:  transport-ws              wss      0      0  0.0.0.0:8089

Endpoint:  302                                          Not in use    0 of inf
    InAuth:  302-auth/302
       Aor:  302                                        1
     Contact:  302/sip:oinku7eu@10.16.20.193:51830;transp a765d330bb NonQual         nan
 Transport:  transport-ws              wss      0      0  0.0.0.0:8089
```

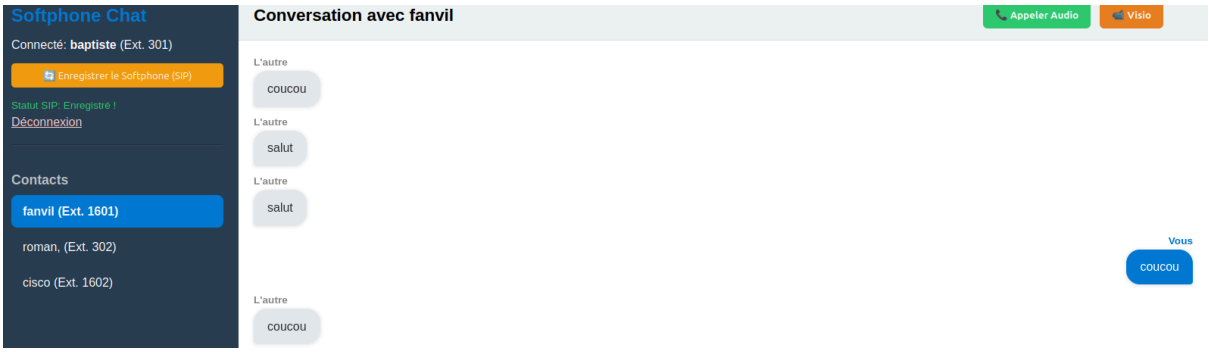
Messagerie instantané :

Pour ce qu'il y est de la messagerie instantanée nous avons fait avec un système de base de données, quand quelqu'un envoie un message ce message est enregistré dans la base de données avec la date et l'heure, le numéro identifiant de l'envoyeur et celui du receveur.

Voici une partie de la table des messages dans la base de données :

```
 21 |          1 |             2 | salut
                     | 2026-01-21 13:49:12 |
 22 |          1 |             2 | test                        | 2026-01-21 13:49:16 |
 23 |          1 |             2 | tu recois bien ?
          | 2026-01-21 13:49:22 |
 24 |          1 |             2 | ça à l'aire de marcher ?*   | 2026-01-21 13:49:34 |
 25 |          3 |             1 | salut
                     | 2026-01-21 13:50:02 |
 26 |          1 |             3 | ça va ?                     | 2026-01-21 13:50:15 |
 27 |          3 |             1 | ça va et toi ?              | 2026-01-21 13:50:22 |
 28 |          3 |             1 | oui
                       | 2026-01-21 13:51:14 |
 29 |          3 |             1 | ça marche bien*             | 2026-01-21 13:51:20 |
 30 |          3 |             1 | instant                     | 2026-01-21 13:51:24 |
 31 |          3 |             1 | coucou                      | 2026-01-21 14:38:42 |
```

Voici ce que nous avons sur l'interface, nous pouvons voir que ici le compte baptiste a discuté avec le compte fanvil.

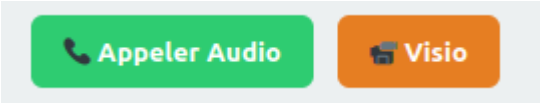


Voici une vidéo qui montre que les messages sont bien instantanés.

https://drive.google.com/file/d/1myjtayWIuNEQsNXk_xs-wC5T6-ln-pHT/view?usp=sharing

Partie Vidéo :

Nous avons essayé de faire en sorte de pouvoir faire des visios grâce à ce bouton :



Si cela était possible quand nous appuyons sur ce bouton une interface avec la caméra ce lancerait et l'appel partirait en même temps.
Mais comme nous n'avons pas les autorisations nécessaires pour utiliser les caméras ce n'est pas possible.

Appel du Softphone vers le téléphone Cisco

Nous avons configuré les appels du SoftPhone vers les téléphones physiques. Comme les extensions SoftPhone sont enregistrés sur le serveur Asterisk nous n’avons qu’à lancer un appel vers le téléphone. Voici une vidéo qui montre l’appel du SoftPhone vers un téléphone physique.

https://drive.google.com/file/d/1Dd4ZWmK0q6PC3iVi9fm2dZZoARUEAcL5/view?usp=sharing

Et voici des captures d’écran des trames capturées quand cet appel est lancé.

162	20.215450742	10.16.20.193	10.15.250.192	TLSv1.2	96	Application Data
163	20.215487320	10.16.20.193	10.15.250.192	TLSv1.2	1922	Application Data
164	20.216927120	10.15.250.192	10.16.20.193	TCP	66	8089 → 52110 [ACK] Seq=29 Ack=1919 Win=496 Len=0 TSval=2716715723 TSecr=106…
165	20.217440707	10.15.250.192	10.16.20.193	TLSv1.2	561	Application Data
166	20.217456464	10.16.20.193	10.15.250.192	TCP	66	52110 → 8089 [ACK] Seq=1919 Ack=524 Win=501 Len=0 TSval=1061870237 TSecr=27…
167	20.221961846	10.16.20.193	10.15.250.192	TLSv1.2	497	Application Data
168	20.225035930	10.16.20.193	10.15.250.192	TLSv1.2	96	Application Data
169	20.225069385	10.16.20.193	10.15.250.192	TLSv1.2	2194	Application Data
170	20.225881568	10.15.250.192	10.16.20.193	TCP	66	8089 → 52110 [ACK] Seq=524 Ack=2380 Win=501 Len=0 TSval=2716715733 TSecr=10…
171	20.226838825	10.15.250.192	10.16.20.193	TCP	66	8089 → 52110 [ACK] Seq=524 Ack=4508 Win=501 Len=0 TSval=2716715734 TSecr=10…
172	20.227996696	10.15.250.192	10.16.20.193	TLSv1.2	390	Application Data
173	20.230032392	10.15.250.192	10.16.20.193	TCP	398	5038 → 41178 [PSH, ACK] Seq=1 Ack=1 Win=510 Len=332 TSval=2716715737 TSecr=…
174	20.230058734	10.16.20.193	10.15.250.192	TCP	66	41178 → 5038 [ACK] Seq=1 Ack=333 Win=501 Len=0 TSval=1061870250 TSecr=27167…
175	20.230032723	10.15.250.192	10.16.20.193	TCP	398	5038 → 41164 [PSH, ACK] Seq=1 Ack=1 Win=510 Len=332 TSval=2716715737 TSecr=…
176	20.230082452	10.16.20.193	10.15.250.192	TCP	66	41164 → 5038 [ACK] Seq=1 Ack=333 Win=501 Len=0 TSval=1061870250 TSecr=27167…
177	20.233152787	10.15.250.192	10.16.20.193	TCP	411	5038 → 41178 [PSH, ACK] Seq=333 Ack=1 Win=510 Len=345 TSval=2716715740 TSec…
178	20.233176790	10.16.20.193	10.15.250.192	TCP	66	41178 → 5038 [ACK] Seq=1 Ack=678 Win=501 Len=0 TSval=1061870253 TSecr=27167…
179	20.233153027	10.15.250.192	10.16.20.193	TCP	411	5038 → 41164 [PSH, ACK] Seq=333 Ack=1 Win=510 Len=345 TSval=2716715740 TSec…
180	20.233211151	10.16.20.193	10.15.250.192	TCP	66	41164 → 5038 [ACK] Seq=1 Ack=678 Win=501 Len=0 TSval=1061870253 TSecr=27167…
181	20.233895093	10.15.250.192	10.16.20.193	TCP	455	5038 → 41164 [PSH, ACK] Seq=678 Ack=1 Win=510 Len=389 TSval=2716715741 TSec…
182	20.233906994	10.16.20.193	10.15.250.192	TCP	66	41164 → 5038 [ACK] Seq=1 Ack=1067 Win=501 Len=0 TSval=1061870253 TSecr=2716…
183	20.233895183	10.15.250.192	10.16.20.193	TCP	455	5038 → 41178 [PSH, ACK] Seq=678 Ack=1 Win=510 Len=389 TSval=2716715741 TSec…
184	20.233929459	10.16.20.193	10.15.250.192	TCP	66	41178 → 5038 [ACK] Seq=1 Ack=1067 Win=501 Len=0 TSval=1061870253 TSecr=2716…
185	20.234038709	10.15.250.192	10.16.20.193	TCP	403	5038 → 41178 [PSH, ACK] Seq=1067 Ack=1 Win=510 Len=337 TSval=2716715741 TSe…

2261	36.877383032	10.16.20.193	10.15.250.192	UDP	224	39517 → 13356 Len=182
2262	36.891709743	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182
2263	36.904626952	10.16.20.193	10.15.250.192	UDP	224	39517 → 13356 Len=182
2264	36.904680116	10.16.20.193	10.15.250.192	UDP	224	39517 → 13356 Len=182
2265	36.912053947	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182
2266	36.932169991	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182
2267	36.936560617	10.16.20.193	10.15.250.192	UDP	224	39517 → 13356 Len=182
2268	36.952055140	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182
2269	36.963339167	10.16.20.193	10.15.250.192	UDP	224	39517 → 13356 Len=182
2270	36.972196285	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182
2271	36.989935078	10.16.20.193	10.15.250.192	UDP	224	39517 → 13356 Len=182
2272	36.991937900	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182
2273	37.012171171	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182
2274	37.015279263	10.16.20.193	10.15.250.192	UDP	224	39517 → 13356 Len=182
2275	37.032295108	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182
2276	37.043578979	10.16.20.193	10.15.250.192	UDP	224	39517 → 13356 Len=182
2277	37.049012364	10.16.20.193	10.15.250.192	UDP	224	39517 → 13356 Len=182
2278	37.052137812	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182
2279	37.055824088	10.15.250.192	10.16.20.193	UDP	156	13356 → 39517 Len=114
2280	37.056441926	10.15.250.192	10.16.20.193	TCP	786	5038 → 41178 [PSH, ACK] Seq=14067 Ack=1 Win=510 Len=720 TSval=2716788155 TSecr=…
2281	37.056487158	10.16.20.193	10.15.250.192	TCP	66	41178 → 5038 [ACK] Seq=1 Ack=14787 Win=501 Len=0 TSval=1061942704 TSecr=2716788…
2282	37.056442072	10.15.250.192	10.16.20.193	TCP	786	5038 → 41164 [PSH, ACK] Seq=14067 Ack=1 Win=510 Len=720 TSval=2716788155 TSecr=…
2283	37.056546557	10.16.20.193	10.15.250.192	TCP	66	41164 → 5038 [ACK] Seq=1 Ack=14787 Win=501 Len=0 TSval=1061942704 TSecr=2716788…
2284	37.056702558	10.15.250.192	10.16.20.193	TCP	785	5038 → 41178 [PSH, ACK] Seq=14787 Ack=1 Win=510 Len=719 TSval=2716788155 TSecr=…
2285	37.056710093	10.16.20.193	10.15.250.192	TCP	66	41178 → 5038 [ACK] Seq=1 Ack=15506 Win=501 Len=0 TSval=1061942704 TSecr=2716788…
2286	37.056702643	10.15.250.192	10.16.20.193	TCP	785	5038 → 41164 [PSH, ACK] Seq=14787 Ack=1 Win=510 Len=719 TSval=2716788155 TSecr=…
2287	37.056740945	10.16.20.193	10.15.250.192	TCP	66	41164 → 5038 [ACK] Seq=1 Ack=15506 Win=501 Len=0 TSval=1061942704 TSecr=2716788…
2288	37.072193713	10.15.250.192	10.16.20.193	UDP	224	13356 → 39517 Len=182

Etude du WebRTC :

Définition : Norme Open Source permettant des communications temps réel (Voix, Vidéo, Données) directement entre navigateurs, sans plugin.

Architecture P2P (Peer-to-Peer) : Les flux média passent directement d'un PC à l'autre pour réduire la latence.

Les 3 APIs Clés :

1. `getUserMedia` : Accès caméra et micro.
2. `RTCPeerConnection` : Gestion de la connexion et du chiffrement.
3. `RTCDataChannel` : Transfert de données (Chat, fichiers).

Sécurité Native : Chiffrement obligatoire (SRTP/DTLS).

Contrainte Majeure : Nécessite impérativement un contexte sécurisé HTTPS. C'est ce qui a bloqué nos tests en local (HTTP), empêchant l'activation des périphériques.

Conclusion

Ce projet avait pour objectif de mettre en place une solution de téléphonie IP centralisée autour d'Asterisk, pilotable à la fois par des téléphones physiques et par une interface web, afin de mieux comprendre l'interconnexion entre les mondes réseau, téléphonie et web.

Nous avons déployé un serveur Asterisk jouant le rôle de PABX IP, capable de gérer des téléphones SIP matériels (Fanvil) ainsi qu'un softphone WebRTC accessible depuis un navigateur. Les utilisateurs SIP ont été définis dans `pjsip.conf`, ce qui permet à Asterisk d'authentifier les terminaux, de connaître leur localisation et de gérer correctement les appels via le dialplan.

Un serveur web a également été intégré au projet. Grâce à l'AMI (Asterisk Manager Interface), ce serveur peut piloter Asterisk à distance, notamment pour déclencher des appels depuis une page web. Cette approche met en évidence la séparation des rôles :

- SIP / PJSIP pour le transport de la voix,
- WebRTC pour l'intégration du téléphone dans un navigateur,
- AMI pour le contrôle et l'automatisation de la téléphonie,
- HTTP/HTML/JavaScript pour l'interface utilisateur.

La mise en œuvre du WebRTC a permis d'aborder des notions avancées telles que WSS, TLS, DTLS et SRTP, indispensables pour sécuriser les communications voix dans un navigateur. Les difficultés rencontrées (certificats, SRTP, compatibilité Alpine Linux) ont permis de mieux comprendre les exigences de sécurité et les dépendances logicielles nécessaires au bon fonctionnement d'une solution de téléphonie moderne.

En conclusion, ce projet a permis de :

- comprendre l'architecture complète d'une solution de téléphonie IP,
- mettre en relation des technologies hétérogènes (web, SIP, sécurité, réseau),
- expérimenter des problématiques réelles de déploiement et de sécurité,
- et concevoir une solution fonctionnelle, évolutive et proche de cas d'usage professionnels (centre d'appels, CRM, supervision).

Ce travail constitue une base solide pour des évolutions futures, comme l'ajout de statistiques d'appels, l'enregistrement des communications, ou l'intégration avec d'autres applications métiers.